

add.txt

01: 403D
02: 4440
03: 0000
04: 000F
05: 7C00
06: 03FF
07: 0400
08: 000A
09: 0001
0A: FFFF
0B: 0800
10: 8101
11: 8202
12: 8304
13: 6413
14: 6523
15: 8605
16: 3716
17: 3826
18: 8908
19: 6779
1a: 6889
1b: 8A06
1c: 3B1A
1d: 3C2A
1e: 8D07
1f: 1BB0
20: 1CCD
21: 2178
22: C12D
23: D12B
24: 8E0A
25: 411E
26: 8209
27: 1112
28: 6BB1
29: 1771
2a: C02D
2b: 6CC1
2c: 1881
2d: 11BC
2e: 820B
2f: 3221
30: C234
31: 7301
32: 1773
33: 6113
34: 411D
35: 5779

36: 4117
37: 9103
38: 0000

sub.txt

00: 4440
01: c03D
02: 0000
03: 8000
04: 7C00
05: 03FF
06: 0400
07: 0800
08: 0001
09: 000F
0A: 000A
0B: 0000
0C: 0000
0D: 0000
0E: 0000
0F: 0000

10: 7B01
11: 8100
12: 8201
13: 8303
14: 3413
15: 8309
16: 6443
17: 8304
18: 3613
19: 830A
1A: 6663
1B: 8305
1C: 3813
1D: 8306
1E: 1883
1F: 8303
20: 3523
21: 8309
22: 6553
23: 8304
24: 3723
25: 830A
26: 6773
27: 8305
28: 3923
29: 8306
2A: 1993
2B: 2A67
2C: CA34
2D: DA36
2E: 2A76
2F: 1C07

30: CA3B
31: 688B
32: 2AAB
33: C030
34: 1C06
35: C03B
36: 1C06
37: CA3B
38: 699B
39: 2AAB
3A: C037
3B: 2E45
3C: CE49
3D: 2E89
3E: CE46
3F: DE43
40: 2D98
41: 1205
42: C051
43: 2D89
44: 1204
45: C051
46: 7D00
47: 1204
48: C051
49: 1D89
4A: 1204
4B: 8307
4C: 3ED3
4D: CE59
4E: 6DDB
4F: 1CCB
50: C059
51: 8306
52: 3FD3
53: CF55
54: C059
55: CD59
56: 5DDB
57: 2CCB
58: C052
59: 8305
5A: 3DD3
5B: 830A
5C: 5CC3
5D: 1ECD
5E: 8309
5F: 5223
60: 1EE2
61: 9E02

62: 0000

Floating Point In TOY

16-bit format · 5-step algorithm using TOY integer operations

1. Extraction

Split the number into sign, exponent, mantissa; restore the implicit leading 1.

2. Alignment

Compare exponents; shift the smaller mantissa right until both are on the same scale.

3. Arithmetic

Same sign → add mantissas. Different sign → subtract smaller from bigger, keep bigger's sign.

4. Normalization

Too big → shift right, exponent +1. Too small → shift left, exponent -1.

5. Packing

Combine the final sign, exponent, and mantissa back into one 16-bit number.

```
// Extraction
unpack A -> signA, expA, mantA (add implicit bit)
unpack B -> signB, expB, mantB (add implicit bit)

// Alignment
diff = expA - expB
IF diff > 0: mantB >>= diff, resultExp = expA
ELSE:      mantA >>= -diff, resultExp = expB

// Arithmetic
IF signA == signB:
    resultMant = mantA + mantB
    resultSign = signA
ELSE:
    resultMant = |mantA - mantB|
    resultSign = sign of bigger mantissa

// Normalization
WHILE resultMant overflows: shift right, resultExp += 1
WHILE resultMant missing leading 1: shift left, resultExp -= 1

// Packing
RETURN combine(resultSign, resultExp, resultMant without implicit bit)
```

Pseudocode